PACKT PUBLISHING



掌握和使用正确的数据可视化方法

Python 数据可视化编程实战

Python Data Visualization Cookbook

[爱尔兰] Igor Milovanović 著
颛清山 译

人民邮电出版社
POSTS & TELECOM PRESS



Python
数据可视化编程实战

[爱尔兰] Igor Milovanović 著
颛清山 译

人 民 邮 电 出 版 社
北 京

图书在版编目（CIP）数据

Python数据可视化编程实战 /（爱尔兰）米洛万诺维奇（Milovanovic,I.）著 ; 颛清山译. -- 北京 : 人民邮电出版社, 2015.5
ISBN 978-7-115-38439-3

Ⅰ. ①P… Ⅱ. ①米… ②颛… Ⅲ. ①软件工具－程序设计 Ⅳ. ①TP311.56

中国版本图书馆CIP数据核字(2015)第057566号

版权声明

Copyright ©2013 Packt Publishing. First published in the English language under the title *Python Data Visualization Cookbook.*
All rights reserved.
本书由英国 Packt Publishing 公司授权人民邮电出版社出版。未经出版者书面许可，对本书的任何部分不得以任何方式或任何手段复制和传播。
版权所有，侵权必究。

◆ 著 [爱尔兰] Igor Milovanović
译 颛清山
责任编辑 陈冀康
责任印制 张佳莹 焦志炜

◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路 11 号
邮编 100164 电子邮件 315@ptpress.com.cn
网址 http://www.ptpress.com.cn
三河市海波印务有限公司印刷

◆ 开本：800×1000 1/16
印张：16.75
字数：318 千字 2015 年 5 月第 1 版
印数：1－3 000 册 2015 年 5 月河北第 1 次印刷
著作权合同登记号 图字：01-2013-9037 号

定价：49.00 元

读者服务热线：(010)81055410 印装质量热线：(010)81055316
反盗版热线：(010)81055315

内容提要

本书是一本使用 Python 实现数据可视化编程的实战指南，介绍了如何使用 Python 最流行的库，通过 60 余种方法创建美观的数据可视化效果。

全书共 8 章，分别介绍了准备工作环境、了解数据、绘制并定制化图表、学习更多图表和定制化、创建 3D 可视化图表、用图像和地图绘制图表、使用正确的图表理解数据以及更多的 matplotlib 知识。

本书适合那些对 Python 编程有一定基础的开发人员阅读，可以帮助读者从头开始了解数据、数据格式、数据可视化，并学会使用 Python 可视化数据。

译者序

图形可视化是展示数据的一个非常好的手段，好的图表自己会说话。毋庸多言，在Python的世界里，matplotlib是最著名的绘图库，它支持几乎所有2D绘图和部分3D绘图，被广泛地应用在科学计算和数据可视化领域。但是介绍matplotlib的中文书籍很少，大部分书籍只是在部分章节中提到了matplotlib的基本用法，因此在内容和深度上都力有不逮。本书则是一本专门介绍matplotlib的译著。

matplotlib是一个开源项目，由John Hunter发起。关于matplotlib的由来，有一个小故事。John Hunter和他研究癫痫症的同事借助一个专有软件做脑皮层电图分析，但是他所在的实验室只有一份该电图分析软件的许可。他和许多一起工作的同事不得不轮流使用该软件的硬件加密狗。于是，John Hunter便有了开发一个工具来替代当前所使用的软件的想法。当时MATLAB被广泛应用在生物医学界中，John Hunter等最初是想开发一个基于MATLAB的版本，但是由于MATLAB的一些限制和不足，加上他本身对Python非常熟悉，于是就有了matplotlib的诞生。

所以，无论从名字上，还是从matplotlib提供的函数名称、参数及使用方法都与MATLAB非常相似。对于一个MATLAB开发人员，使用起来会相当得心应手。即使对不熟悉MATLAB的开发人员（譬如我），对其函数的使用也能够一目了然，而且matplotlib有着非常丰富的文档和实例，加上本书的介绍，学习起来将会非常轻松。

matplotlib命令提供了交互绘图的方式，在Python的交互shell中，我们可以执行matplotlib命令来实时地绘制图形并对其进行修改。生成的图像可以保存成许多格式，这取决于其所使用的后端，但绝大多数后端都支持如png、pdf、ps、eps和svg等格式。

在之前的项目中，我使用 Python 的 Locust 工具进行性能测试，该工具非常出色，然而在对获取到的性能数据的分析上，没有提供太多的功能。于是我决定使用 matplotlib 进行性能数据的分析和可视化。从绘制最简单的柱状图、线形图，到引入散点图、直方图，我渐渐对 matplotlib 有了进一步的了解，也对它提供了如此强大的功能却又不失易用性而着迷。

虽然本书是一本 cookbook，然而它并不仅仅局限在讲解如何绘制各种图形上，更重要的是，本书让我们了解了如何用正确的图形把数据可视化出来，也就是“do the right thing in the right way”。在翻译本书的过程中，我意识到，如果我当时手头有这么一本书，将会少走不少弯路。本书包括了非常多的图形介绍以及丰富的示例，我相信，读完本书以后，读者将能应对各种常见的数据可视化问题。

在这里，我要特别感谢一下我的妻子董秋影，在精神和专业知识上她都给予了我莫大的帮助，没有她就没有这本译稿的完成。她从事医疗图像算法工作，对各种图形和算法以及 MATLAB 都有很深的了解，本书的每一章都经过了她认真的审阅校对。此外，感谢彭明伟先生完成了第 1 章的初稿翻译工作。最后，感谢人民邮电出版社陈冀康老师专业细心地审核，和陈老师合作很轻松、很开心。

由于译者水平有限，错误和失误在所难免，如有任何意见和建议，请不吝指正，我将感激不尽。我的邮箱：zhuanqingshan@163.com。

颛清山

2014 年 12 月于北京

作者简介

Igor Milovanović 是一个在Linux系统和软件工程领域有深厚背景的经验丰富的开发人员。具备创建可扩展数据驱动分布式富软件系统的技术。

他是一个高性能系统设计的布道者，对软件架构和开发方法论有着浓厚的兴趣。他一直坚持倡导促进高质量软件的方法论，如测试驱动开发、一键部署和持续集成。

他也拥有坚实的产品开发知识。拥有领域经验知识，并参加过官方培训，他能够在业务和开发人员之间很好地传递业务知识和业务流程。

非常感谢我的未婚妻，她允许我把大量的时间花费在工作上而没有陪伴她，并在我无休止地谈论本书时甘愿做一个热心的听众。我也想感谢我的哥哥，他一直是我坚强的后盾。感谢我的父母，给予我各种发展自己的空间，让我成为今天的自己。

如果没有开发Python、matplotlib和所有本书中使用的库的开源社区的巨大能量，我不可能写出这本书。我深深地感谢所有这些项目背后的人们。感谢你们！

评阅者简介

Tarek Amr 从东安格里亚大学获得了数据挖掘和信息检索专业的研究生学位。他在软件开发领域有近 10 年的经验。自从 2007 年开始，他一直在 Global Voices Online (GVO)义务工作，目前他是埃及 Open Knowledge Foundation (OKFN)的大使。他热衷于开放数据、Government 2.0、数据可视化、数据新闻、机器学习和自然语言处理。

Tarek 的 Twitter 账号是@gr33ndata，主页是 http://tarekamr.appspot.com/。

Jayesh K. Gupta 是 Matlab Toolbox for Biclustering Analysis (MTBA)的首席开发人员。他目前是一名 IIT Kanpur 的在读研究生和研究员。他的兴趣是模式识别。他对基础科学也有浓厚的兴趣，认为它们是自然界中的模式分析工具。来到 IIT 之后，他看到这种分析是如何借助机器学习算法广泛应用在各种不同的应用程序中的。他相信通过机器智能来强化人类的想法是增进人类知识的最好方式之一。他是一个长期的技术爱好者和自由软件的布道者。他的网名是 rejuvyesh。他也是一名狂热的读者，从 Goodreads 可以获得他读过的书籍的信息。从 Bitbucket 和 GitHub 可以找到他的项目。所有的链接都在 http://home.iitk.ac.in/~jayeshkg/上，也可以通过 a2z.jayesh@gmail.com 联系他。

Kostiantyn Kucher 出生在乌克兰敖德萨。2012 年他在敖德萨国立理工大学获得了计算机科学专业的硕士学位。他使用 Python、Matplotlib 和 PIL 从事机器学习和图像识别的工作。

目前，Kostiantyn 是一名计算机科学专业信息可视化方向的博士研究生。他在 Andreas Kerren 博导的指导下，在瑞典林奈大学计算机科学系的 ISOVIS 小组进行研究。

Kenneth Emeka Odoh 从事高级的数据可视化技术研究工作。他的研究兴趣是通过可

视的线索指导用户得出研究结果的探索性研究。

Kenneth 精通 Python 编程。2012 年他曾在芬兰的 Pycon 大会做演讲，主题是 Django 中的数据可视化。

他目前是加拿大里贾纳大学的一名研究员，通晓多种编程语言，有 C、C++、Python 和 Java 的应用开发经验。

编写代码之余，Kenneth 还参加了坎皮恩学院圣歌合唱团。

前言

最好的数据是我们能看到并理解的数据。作为一个开发人员，我们想创造并构建出最全面且容易理解的可视化图形。然而这并非总是很简单，我们需要找出数据，读取它、清理它、揣摩它，然后使用恰当的工具将其可视化。本书通过简单（和不那么简单）直接的方法解释了如何读取、清理和可视化数据的流程。

本书对怎样读取本地数据、远程数据、CSV、JSON 以及关系型数据库中的数据，都进行了讲解。

通过 matplotlib，我们能用一行简单的 Python 代码绘制出一些简单的图表，但是进行更高级的绘图还需要除 Python 之外的其他知识。我们需要理解信息理论和人类的审美学来生成最吸引人的可视化效果。

本书讲解在 Python 中使用 matplotlib 绘图的一些练习、使用情况，以及对于不同图表特性应该使用的方法的一些最佳实践。

本书的写作及代码开发均基于 Ubuntu 12.03，使用了 Python 2.7、IPython 0.13.2、virtualenv 1.9.1、matplotlib 1.2.1、NumPy 1.7.1 和 SciPy 0.11.0。

本书涵盖内容

第 1 章，准备工作环境，包括一些安装方法，以及如何在你的平台上安装所需的 Python 包和库的一些建议。

第 2 章，了解数据，介绍通用的数据格式，以及如何读写，如 CSV、JSON、XSL 或

者关系型数据库。

第 3 章，绘制图表及定制化，着手绘制简单的图表并介绍图表的定制化。

第 4 章，学习更多图表和定制化，继续上一章内容，介绍更多的高级表格和网格定制化。

第 5 章，3D 可视化，介绍三维数据的可视化，如 3D 柱状图、3D 直方图，以及 matplotlib 动画。

第 6 章，用图像和地图绘制图表，涵盖图像处理、在地图上投射数据，以及创建 CAPTCHA 测试图像。

第 7 章，使用正确的图表理解数据，涵盖一些更高级绘图技术的讲解和方法，如频谱图和相关性。

第 8 章，更多的 matplotlib 知识，介绍一些图表如甘特图、箱线图，并且介绍如何在 matplotlib 中使用 LaTeX 渲染文本。

准备工作

学习本书时，需要你在自己的操作系统上安装 Python2.7.3 或最新版本。本书使用 Ubuntu12.03 系统上的默认 Python 版本（2.7.3）。

本书中用到的另一个软件包是 IPython，它是一个交互式的 Python 环境，功能非常强大、灵活。可以通过基于 Linux 平台的包管理工具或者用于 Windows 和 Mac OS 系统的预安装文件安装。

一般来说，如果你对于 Python 安装和相关软件安装不熟悉，强烈推荐你使用预打包的 Python 科学发行包如 Anaconda、Enthought Python 发行包或者 Python（X,Y）进行安装。

其他所需的软件主要包括 Python 包，可全部通过 Python 安装管理器 pip 进行安装。pip 本身通过 Python 的 easy_install 安装工具安装。

谁适合阅读本书

本书是为那些通常已经了解 Python 编程的开发人员编写的。如果你听说过数据可视化但又不知道从何入手，本书会从头开始指导你了解数据、数据格式、数据可视化，以及如

何使用 Python 可视化数据。

你需要知道一些一般的编程概念，如果你有编程经验，会非常有用。然而，本书中的代码几乎是逐行讲解的。阅读本书不需要任何数学知识，书中介绍的每一个概念都有详细的讲解，并且提供了一些参考资料以供进一步的兴趣阅读。

约定

在本书中，不同的信息由一些不同风格的文字来区分。这里有一些文字风格的例子，以及它们的含义解释。

书中的代码文字显示如下："我们把小演示程序封装在 `DemoPIL` 类中，这样可以共享示例函数 `run_fixed_filters_demo` 的代码，并能很容易地对其进行扩展。"

代码块设置如下：

```
def _load_image(self, imfile):
    self.im = mplimage.imread(imfile)
```

当我们想要让你关注代码块中的某一特定部分时，相关的行或元素将设置为粗体：

```
# tidy up tick labels size
all_axes = plt.gcf().axes
for ax in all_axes:
    for ticklabel in ax.get_xticklabels() + ax.get_yticklabels():
        ticklabel.set_fontsize(10)
```

所有的命令行输入或者输出的写法如下。

```
$ sudo python setup.py install
```

新术语和**关键词**将显示为粗体。例如，在屏幕上、菜单或对话框中的文字将会显示为："然后我们为火柴杆图设置一个标签和基线位置，默认值为 **0**。"



警告或者重要的说明出现在这样的一个文本框中。



提示和技巧像这样显示。

读者反馈

欢迎读者向我们反馈意见。请让我们知道你对本书的看法——哪些是你喜欢或者不喜欢的。读者反馈对我们非常重要，可以帮我们完善一些你非常关心的内容。

如果给我们发送一般的反馈，可以简单地发电子邮件到 feedback@packtpub.com，并请在消息标题中提及书名。

如果你对某个话题有经验，并且有兴趣写作或者想为一本书做贡献，请参考我们的作者指南 www.packtpub.com/authors。

支持

既然你已经是 Packt 书籍的读者，我们有许多辅助材料可以帮助你从本书中得到最大的收获。

下载示例代码

可以在 http://www.packtpub.com 网站上你的账户中下载你所购买的所有图书的示例代码文件。如果你在其他地方购买本书，可以访问 http://www.packtpub.com/support 页面进行登记，文件会通过邮件直接发送给你。

勘误

尽管我们已经竭尽全力确保本书内容的准确，但是错误在所难免。如果你在书中发现了错误——可能是文本或者代码中的错误——如果你能把它报告给我们，我们将万分感谢。如此，这样可以减轻其他读者的痛苦，并且可以帮我们改进该书的后续版本。如果你找到任何错误，请访问 http://www.packtpub.com/submit-errata 并报告给我们，选择你的图书，点击 **errata submission form** 链接，并加入你勘误的详细内容。一旦你的勘误通过验证，你的提交将被接受，勘误将会上传到我们的网站，或者添加到位于该标题的勘误部分的已有勘误列表中。可以在 http://www.packtpub.com/support 上选择你的标题来查看所有现有的勘误信息。

著作权侵害

互联网上的版权侵害是一个跨越所有媒介的持续的问题。在 Packt，我们很认真地看待版权和许可保护。如果你不经意在互联网上得到了关于我们作品的任何形式的不合法的副

本，请及时给我们提供其地址或者网站名称，以便我们及时补救。

请通过 copyright@packtpub.com 联系我们，同时请提供涉嫌侵权材料的链接。

非常感激你帮助保护我们的作者，让我们尽力提供更有价值的内容。

问题

如果你对本书有任何疑问，可以通过 questions@packtpub.com 联系我们，我们会竭尽全力提供帮助。

目录

第 1 章　准备工作环境 1

1.1　介绍 1
1.2　安装 matplotlib、Numpy 和 Scipy 库 2
1.2.1　准备工作 2
1.2.2　操作步骤 3
1.2.3　工作原理 4
1.2.4　补充说明 4
1.3　安装 virtualenv 和 virtualenvwrapper 4
1.3.1　准备工作 5
1.3.2　操作步骤 5
1.4　在 Mac OS X 上安装 matplotlib 6
1.4.1　准备工作 6
1.4.2　操作步骤 6
1.5　在 Windows 上安装 matplotlib 7
1.5.1　准备工作 7
1.5.2　操作步骤 8
1.5.3　补充说明 8
1.6　安装图像处理工具：Python 图像库（PIL） 9
1.6.1　操作步骤 9
1.6.2　安装过程说明 9
1.6.3　补充说明 9
1.7　安装 requests 模块 10
1.7.1　操作步骤 10

1.7.2 requests 使用说明 10
1.8 在代码中配置 matplotlib 参数 11
1.8.1 准备工作 11
1.8.2 操作步骤 11
1.8.3 代码解析 12
1.9 为项目设置 matplotlib 参数 12
1.9.1 准备工作 12
1.9.2 配置方法 12
1.9.3 配置过程说明 13
1.9.4 补充说明 14
第 2 章 了解数据 15
2.1 简介 16
2.2 从 CSV 文件导入数据 16
2.2.1 准备工作 16
2.2.2 操作步骤 16
2.2.3 工作原理 17
2.2.4 补充说明 18
2.3 从 Microsoft Excel 文件中导入数据 18
2.3.1 准备工作 19
2.3.2 操作步骤 19
2.3.3 工作原理 19
2.3.4 补充说明 20
2.4 从定宽数据文件导入数据 21
2.4.1 准备工作 21
2.4.2 操作步骤 21
2.4.3 工作原理 22
2.5 从制表符分隔的文件中读取数据 23
2.5.1 准备工作 23
2.5.2 操作步骤 23
2.5.3 工作原理 23
2.5.4 补充说明 24
2.6 从 JSON 数据源导入数据 24
2.6.1 准备工作 25
2.6.2 操作步骤 25
2.6.3 工作原理 25

2.6.4 补充说明.....26
2.7 导出数据到 JSON、CSV 和 Excel.....27
2.7.1 准备工作.....27
2.7.2 操作步骤.....27
2.7.3 工作原理.....30
2.7.4 补充说明.....31
2.8 从数据库导入数据.....31
2.8.1 准备工作.....32
2.8.2 操作步骤.....32
2.8.3 工作原理.....35
2.8.4 补充说明.....35
2.9 清理异常值.....36
2.9.1 准备工作.....36
2.9.2 操作步骤.....36
2.9.3 补充说明.....42
2.10 读取大块数据文件.....42
2.10.1 操作步骤.....42
2.10.2 工作原理.....43
2.10.3 补充说明.....44
2.11 读取流数据源.....44
2.11.1 操作步骤.....44
2.11.2 工作原理.....45
2.11.3 补充说明.....45
2.12 导入图像数据到 NumPy 数组.....46
2.12.1 准备工作.....46
2.12.2 操作步骤.....46
2.12.3 工作原理.....49
2.12.4 补充说明.....50
2.13 生成可控的随机数据集合.....51
2.13.1 准备工作.....51
2.13.2 操作步骤.....52
2.14 真实数据的噪声平滑处理.....58
2.14.1 准备工作.....58
2.14.2 操作步骤.....58
2.14.3 工作原理.....58

2.14.4 补充说明.....62
第 3 章 绘制并定制化图表.....65
3.1 简介.....65
3.2 定义图表类型——柱状图、线形图和堆积柱状图.....66
3.2.1 准备工作.....66
3.2.2 操作步骤.....66
3.2.3 工作原理.....69
3.2.4 补充说明.....70
3.3 简单的正弦图和余弦图.....71
3.3.1 准备工作.....71
3.3.2 操作步骤.....71
3.4 设置坐标轴长度和范围.....74
3.4.1 准备工作.....74
3.4.2 操作步骤.....74
3.4.3 工作原理.....74
3.4.4 补充说明.....76
3.5 设置图表的线型、属性和格式化字符串.....76
3.5.1 准备工作.....77
3.5.2 操作步骤.....77
3.5.3 工作原理.....77
3.6 设置刻度、刻度标签和网格.....80
3.6.1 准备工作.....80
3.6.2 操作步骤.....81
3.7 添加图例和注解.....83
3.7.1 准备工作.....84
3.7.2 操作步骤.....84
3.7.3 工作原理.....85
3.8 移动轴线到图中央.....86
3.8.1 操作步骤.....86
3.8.2 工作原理.....87
3.8.3 补充说明.....87
3.9 绘制直方图.....87
3.9.1 准备工作.....88
3.9.2 操作步骤.....88

3.9.3 工作原理.....90
3.10 绘制误差条形图.....90
3.10.1 准备工作.....90
3.10.2 操作步骤.....90
3.10.3 工作原理.....91
3.10.4 补充说明.....92
3.11 绘制饼图.....92
3.11.1 准备工作.....92
3.11.2 操作步骤.....93
3.12 绘制带填充区域的图表.....94
3.12.1 准备工作.....94
3.12.2 操作步骤.....94
3.12.3 工作原理.....95
3.12.4 补充说明.....96
3.13 绘制带彩色标记的散点图.....96
3.13.1 准备工作.....96
3.13.2 操作步骤.....96
3.13.3 工作原理.....98
第 4 章 学习更多图表和定制化.....99
4.1 简介.....99
4.2 设置坐标轴标签的透明度和大小.....100
4.2.1 准备工作.....100
4.2.2 操作步骤.....100
4.2.3 工作原理.....101
4.2.4 补充说明.....102
4.3 为图表线条添加阴影.....102
4.3.1 准备工作.....103
4.3.2 操作步骤.....103
4.3.3 工作原理.....105
4.3.4 补充说明.....105
4.4 向图表添加数据表.....106
4.4.1 准备工作.....106
4.4.2 操作步骤.....106
4.4.3 工作原理.....107
4.4.4 补充说明.....107

4.5 使用 subplots(子区) 108
4.5.1 准备工作 108
4.5.2 操作步骤 108
4.5.3 工作原理 110
4.5.4 补充说明 110
4.6 定制化网格 110
4.6.1 准备工作 110
4.6.2 操作步骤 112
4.6.3 工作原理 114
4.7 创建等高线图 114
4.7.1 准备工作 114
4.7.2 操作步骤 115
4.7.3 工作原理 117
4.8 填充图表底层区域 117
4.8.1 准备工作 118
4.8.2 操作步骤 118
4.8.3 工作原理 120
4.9 绘制极线图 121
4.9.1 准备工作 121
4.9.2 操作步骤 121
4.9.3 工作原理 123
4.10 使用极线条可视化文件系统树 123
4.10.1 准备工作 123
4.10.2 操作步骤 123
4.10.3 工作原理 126
第 5 章 创建 3D 可视化图表 129
5.1 简介 129
5.2 创建 3D 柱状图 129
5.2.1 准备工作 130
5.2.2 操作步骤 130
5.2.3 工作原理 132
5.2.4 补充说明 132
5.3 创建 3D 直方图 133
5.3.1 准备工作 134

5.3.2 操作步骤.....134
5.3.3 工作原理.....135
5.4 在 matplotlib 中创建动画.....136
5.4.1 准备工作.....136
5.4.2 操作步骤.....137
5.4.3 工作原理.....138
5.4.4 补充说明.....139
5.5 用 OpenGL 制作动画.....139
5.5.1 准备工作.....140
5.5.2 操作步骤.....141
5.5.3 工作原理.....142
5.5.4 补充说明.....142
第 6 章 用图像和地图绘制图表.....145
6.1 简介.....145
6.2 用 PIL 做图像处理.....146
6.2.1 准备工作.....146
6.2.2 操作步骤.....149
6.2.3 工作原理.....151
6.2.4 补充说明.....151
6.3 绘制带图像的图表.....151
6.3.1 准备工作.....152
6.3.2 操作步骤.....152
6.3.3 工作原理.....154
6.4 在具有其他图形的图表中显示图像.....156
6.4.1 准备工作.....156
6.4.2 操作步骤.....156
6.4.3 工作原理.....158
6.4.4 补充说明.....159
6.5 使用 Basemap 在地图上绘制数据.....160
6.5.1 准备工作.....160
6.5.2 操作步骤.....160
6.5.3 工作原理.....162
6.5.4 补充说明.....164
6.6 使用 Google Map API 在地图上绘制数据.....164
6.6.1 准备工作.....164

6.6.2 操作步骤 165
6.6.3 工作原理 168
6.6.4 补充说明 169
6.7 生成 CAPTCHA 图像 169
6.7.1 准备工作 170
6.7.2 操作步骤 170
6.7.3 工作原理 173
6.7.4 补充说明 174
第 7 章 使用正确的图表理解数据 177
7.1 简介 177
7.2 理解对数图 178
7.2.1 准备工作 178
7.2.2 操作步骤 179
7.2.3 工作原理 180
7.3 理解频谱图 181
7.3.1 准备工作 181
7.3.2 操作步骤 182
7.3.3 工作原理 183
7.3.4 补充说明 184
7.4 创建火柴杆图 185
7.4.1 准备工作 186
7.4.2 操作步骤 186
7.4.3 工作原理 188
7.5 绘制矢量场流线图 189
7.5.1 准备工作 189
7.5.2 操作步骤 189
7.5.3 工作原理 191
7.5.4 补充说明 192
7.6 使用颜色表 192
7.6.1 准备工作 192
7.6.2 操作步骤 194
7.6.3 工作原理 196
7.6.4 补充说明 196
7.7 使用散点图和直方图 197
7.7.1 准备工作 197

7.7.2 操作步骤.....198
7.7.3 工作原理.....201
7.7.4 补充说明.....201
7.8 绘制两个变量间的互相关图形.....204
7.8.1 准备工作.....204
7.8.2 操作步骤.....205
7.8.3 工作原理.....206
7.9 自相关的重要性.....207
7.9.1 准备工作.....207
7.9.2 操作步骤.....207
7.9.3 工作原理.....209
7.9.4 补充说明.....209
第 8 章 更多的 matplotlib 知识.....211
8.1 简介.....211
8.2 绘制风杆（barbs）.....211
8.2.1 准备工作.....212
8.2.2 操作步骤.....213
8.2.3 工作原理.....214
8.2.4 补充说明.....215
8.3 绘制箱线图.....215
8.3.1 准备工作.....215
8.3.2 操作步骤.....216
8.3.3 工作原理.....217
8.4 绘制甘特图.....218
8.4.1 准备工作.....218
8.4.2 操作步骤.....219
8.4.3 工作原理.....222
8.5 绘制误差条.....223
8.5.1 准备工作.....223
8.5.2 操作步骤.....224
8.5.3 工作原理.....225
8.5.4 补充说明.....225
8.6 使用文本和字体属性.....226
8.6.1 准备工作.....226
8.6.2 操作步骤.....229

8.6.3 工作原理.....232
8.7 用 LaTeX 渲染文本.....232
8.7.1 准备工作.....233
8.7.2 操作步骤.....234
8.7.3 工作原理.....236
8.7.4 补充说明.....236
8.8 理解 pyplot 和 OO API 的不同.....237
8.8.1 准备工作.....237
8.8.2 操作步骤.....239
8.8.3 工作原理.....241
8.8.4 补充说明.....242

第 1 章
准备工作环境

本章包含以下内容。

- 安装 matplotlib、NumPy 和 SciPy 库
- 安装 virtualenv 和 virtualenvwrapper
- 在 Mac OS X 上安装 matplotlib
- 在 Windows 上安装 matplotlib
- 安装 Python 图像处理库（Python Imaging Library，PIL）
- 安装 requests 模块
- 通过代码设置 matplotlib 的参数
- 为项目设置 matplotlib 的参数

1.1 介绍

本章向读者介绍必备的工具类库，以及如何进行安装与配置。作为本书后续部分的基础知识，掌握这部分内容十分必要。如果你没有使用 Python 进行数据处理、图像处理以及数据可视化的经验，建议不要跳过本章。如略过本章，在需要安装配套工具软件或需要确定工程所支持的软件版本时，可返回本章阅读相关内容。

1.2 安装 matplotlib、Numpy 和 Scipy 库

本章介绍了 **matplotlib** 及其依赖的软件在 Linux 平台上的几种安装方法。

1.2.1 准备工作

这里假设你已经安装了 Linux 系统且安装好了 Python（推荐使用 Debian/Ubuntu 或 RedHat/SciLinux）。在前面提到的Linux 系统发行版中，Python 通常是默认安装的。如果没有，使用标准的软件安装方式安装 Python 也是非常简便的。本书假设你安装的 Python 版本为 2.7 或以上。



几乎所有的代码均可在 Python 3.3 及以上版本的环境下工作，但是因为大部分操作系统提供的 Python 版本仍然是 2.7（甚至是 2.6），本书代码基于 Python 2.7 版本。这种基于 Python 版本的区别并不大，主要是在软件包版本和部分代码上存在差别（在 Python3.3 以上版本，请使用 range 方法替换 xrang 方法）。

本书也假设你知道如何使用操作系统软件包管理工具进行软件包的安装，以及知道如何使用命令行终端。

构建 matplotlib 运行环境，需要满足相关软件依赖。

Matplotlib 的构建过程依赖 **NumPy**、**libpng** 和 **freetype** 软件包。要从源代码构建 matplotlib，必须先要安装好 NumPy 库。读者可以访问 http://www.numpy.org/了解安装 NumPy 库的方法（请安装 1.4 或以上版本，Python 3 需要 NumPy 1.5 或以上版本）。



NumPy 库提供处理大数据集的数据结构和数学方法。诸如元组、列表或字典等 Python 的默认数据结构同样可以很好地支持数据的插入、删除和连接。NumPy 的数据结构支持“矢量”操作，使用简便，同时具有很高的执行效率。矢量操作在实现时充分考虑了大数据的需要，基于 C 语言的实现方式也保证了执行效率。

基于 NumPy 构建的 SciPy 库，是 Python 的标准科学计算和数学计算工具包，包含了大量的专用函数和算法。而大部分函数和算法源自著名的 Netlib 软件仓库（参见 http://www.netlib.org），实际上是使用 C 语言和 Fortran 语言实现的。

安装 NumPy 库的步骤如下。

1. 安装 Python-NumPy 软件包。

```
$ sudo apt-get install python-numpy
```

2. 检查软件包版本。

```
$ python -c 'import numpy; print numpy.__version__'
```

3. 安装所需的库。

- **libpng 1.2**：PNG 文件处理（依赖 zlib 库）。
- **freetype 1.4+**：处理 True type 字体。

```
$ sudo apt-get install build-dep python-matplotlib
```

如果使用 RedHat 或基于 RedHat 的 Linux 发行版（Fedora、SciLinux 或 CentOS），可以使用 yum 工具进行安装，方法与 apt-get 工具类似。

```
$ su -c 'yum-builddep python-matplotlib'
```

1.2.2 操作步骤

安装 matplotlib 及其依赖软件的方法有很多：从源代码安装，使用预编译完成的二进制文件安装，通过操作系统软件包管理工具安装，或安装内置了 matplotlib 的 python 预打包发布版本。

使用包管理工具大概是最简单的安装方式。例如在 Ubuntu 系统中，在命令行终端中输入下面的命令即可。

```
# in your terminal, type:
$ sudo apt-get install python-numpy python-matplotlib python-scipy
```

如果读者期望使用最新特性，最好的选择是通过源代码进行安装。安装方式包含以下步骤：获取源代码、构建依赖库和参数配置、编译以及安装。

可以从代码托管站点 www.github.com 下载最新代码进行安装，操作步骤如下。

```
$ cd ~/Downloads/
$ wget https://github.com/downloads/matplotlib/matplotlib/matplotlib-1.2.
0.tar.gz
$ tar xzf matplotlib-1.2.0.tar.gz
$ cd matplotlib-1.2.0
$ python setup.py build
$ sudo python setup.py install
```



下载示例代码

对于使用网站账户在 http://www.packtpub.com 上购买的所有 Packt 书籍，读者均可在网站上下载有关的代码示例。如果读者是从别处购得图书，可以访问网址（http://www.packtpub.com/support/），完成注册后，代码文件会发送到读者邮箱。

1.2.3 工作原理

从源代码安装 matplotlib，使用了标准的 Python 发布工具 **Distutils**。安装过程需要提前安装依赖的软件包。关于使用标准的 Linux 包管理工具安装依赖软件的方法，可参考本节中关于准备工作的说明。

1.2.4 补充说明

根据数据可视化项目的需要，可能有必要安装额外的可选软件包 。

无论你工作在什么项目上，IPython 都是值得推荐的。IPython 是一款交互式 Python 命令行工具。其提供的 PyLab 模式，已经导入了 matplotlib 库与相关软件包（例如 NumPy 和 SciPy），可以直接使用相关库的功能。IPython 工具的安装与使用方法十分简单明了，读者可通过 IPython 的官方网站查看相关细节。

1.3 安装 virtualenv 和 virtualenvwrapper

如果同时工作在多个项目上，或是需要在不同项目间频繁切换，将所有的软件都安装在操作系统层级上也许不是一个好主意。当需要在不同系统（产品环境）上运行软件时，这种方式会带来问题。如果到此时才发现缺少特定的软件包，或是产品环境已经安装的软件包存在版本冲突，这将是非常痛苦的。为避免这种情况发生，可以选择使用 virtualenv。

virtualenv 是由 Ian Bicking 创建的开放源代码项目。通过这个项目，开发人员可以把不同项目的工作环境隔离开，从而能够更容易地维护多种不同的软件包版本。

举例来说，Django 网站系统是基于 Django 1.1 和 Python 2.3 版本开发的，但与此同时，一个新项目要求必须基于 Python2.6 来开发。在笔者工作过的项目中，根据项目的需要同时使用多个版本的 Python（以及相关软件包）的情况非常普遍。

virtualenv 能够让我们很容易地在不同的运行环境之间切换。同时，如果需要切换到另

外的机器或者需要在产品服务器（或客户的工作站主机）上部署软件，用 virtualenv 能够很容易地重新构建相同的软件包环境。

1.3.1　准备工作

若安装 virtualenv，需要用到 Python 和 pip。Pip 是安装并管理 Python 软件包的工具，可以用它来代替 easy_install 工具。本书中大部分的软件包都是用 pip 工具进行管理的。只需在终端中以 root 身份执行如下命令，就可以很容易地完成 pip 的安装。

```
# easy_install pip
```

virtualenv 本身已经相当不错了，然而如果配合 virtualenvwrapper，一切变得更加简单，并且组织多个虚拟环境的工作也会更加容易。virtualenvwrapper 的功能请参考 http://virtualenvwrapper.readthedocs.org/en/latest/#features。

1.3.2　操作步骤

安装 virtualenv 和 virtualenvwrapper 工具的步骤如下。

1. 安装 virtualenv 和 virtualenvwrapper。

```
$ sudo pip virtualenv
$ sudo pip virtualenvwrapper
# 创建保存虚拟环境的目录，并使用 export 导出为环境变量。
$ export VIRTENV=~/.virtualenvs
$ mkdir -p $VIRTENV
# 使用 source 命令调用（执行）shell 脚本来激活包装器
$ source /usr/local/bin/virtualenvwrapper.sh
# 创建一个虚拟环境
$ mkvirtualenv virt1
```

2. 在 `virt1` 环境中安装 matplotlib。

```
(virt1)user1:~$ pip install matplotlib
```

3. 很有可能需要把以下代码添加到`~/.bashrc` 中。

```
source /usr/loca/bin/virtualenvwrapper.sh
```

下面是一些有用和频繁使用的命令。

- `mkvirtualenv ENV`: 创建名为 ENV 的虚拟环境并激活。
- `workon ENV`: 激活先前创建的 ENV 虚拟环境。
- `deactivate`: 退出当前虚拟环境。

1.4 在 Mac OS X 上安装 matplotlib

在 Mac OS X 上获取 matplotlib 最简便的方式是使用预打包的 python 发布版本，例如 **Enthought Python Distribution (EPD)**。读者可以直接访问 EPD 网站，下载安装操作系统对应的最新稳定版。

倘若 EPD 软件不满足要求，或者因为其他一些原因（如版本问题）而无法使用，也可以用手动（麻烦点）的方式安装 Python、matplotlib 和依赖软件。

1.4.1 准备工作

对于 Apple 在操作系统中没有安装的软件来说，**Homebrew** 项目可以使安装过程更容易。实际上，Homebrew 是基于 Ruby 和 Git 的，可以被自动下载和安装。软件安装顺序为：首先安装 Homebrew,之后安装 Python，随后安装诸如 virtualenv 的工具软件，接下来安装 matplotlib 的依赖（NumPy 和 SciPy），最后安装 matplotlib。接下来就开始吧。

1.4.2 操作步骤

1. 在终端中输入并执行下面的命令。

```
ruby <(curl -fsSkL raw.github.com/mxcl/homebrew/go)
```

命令执行完成后，可以尝试用 brew update 或 brew doctor 命令来检查 brew 是否能够正常工作。

2. 然后，将 Homebrew 目录添加到系统 path 环境变量中。这样，使用 Homebrew 安装的软件包能够获得比其他版本更高的优先级。打开`~/.bash_profile`文件（或者`/Users/[your-user-name]/.bash_profile`）并在文件末尾添加以下代码。

```
export PATH=/usr/local/bin:$PATH
```

3. 重新启动命令行终端使其加载新的 path 环境变量。之后，下面一行简单的代码就可以完成 Python 的安装。

```
brew install python --framework -universal
```

本命令同时也将安装 Python 所需的其他软件。

4. 更新 path 环境变量（添加到同一行）。

```
export PATH=/usr/local/share/python:/usr/local/bin:$PATH
```

5. 在命令行输入 `python -version`，检查 python 是否安装成功。

正常的话，会能够看到 Python 版本信息为 2.7.3。

6. pip 应该也已经安装完毕。如果还没有，可使用 `easy_install` 安装 pip。

```
$ easy_install pip
```

7. 这时，任何所需软件包的安装过程就变得非常简单了。例如，安装 virtualenv 和 virtualenvwrapper。

```
pip install virtualenv
pip install virtualenvwrapper
```

8. 是时候向一直以来的目标迈进了——安装 matplotlib。

```
pip install numpy
brew install gfortran
pip install scipy
```



Mountain Lion 的用户需要安装 SciPy 的开发版（0.11），命令如下。

```
pip install -e git+https://github.com/scipy/scipy#egg=scipy- dev
```

9. 检查安装是否成功。启动 Python 并执行以下命令。

```
import numpy
print numpy.__version__
import scipy
print scipy.__version__
quit()
```

10. 安装 matplotlib。

```
pip install matplotlib
```

1.5　在 Windows 上安装 matplotlib

在本节中，我们将演示如何安装 Python 和 matplotlib。假设系统中没有预先安装 Python。

1.5.1　准备工作

在 Windows 上安装 matplotlib 有两种方式。较简单的方式是安装预打包的 Python 环境，如 EPD、Anaconda 和 Python(x,y)。这是本书推荐的安装方式，尤其对于初学者来说更是如此。

第二种方式，是使用预编译的二进制文件来安装 matplotlib 和依赖软件包。需要注意安装的 NumPy 和 SciPy 的版本，因为并非所有的版本都与最新版 matplotlib 二进制文件相互兼容，这势必会给整个安装过程带来一些困难。这种安装方法也有自身的优势。如果想要获取最新功能，即使功能还未正式发布，仍然能够通过编译 matplotlib 或某软件库的某个特定版本来使用它。

1.5.2 操作步骤

要安装免费或商业 Python 科学发布版，按照项目网站上提供的步骤可以很容易安装成功，这也是推荐使用的方式。

如果单纯使用 matplotlib,不期望面对 Python 和依赖软件包版本所带来的困扰，可以考虑使用 Enthought Python Distribution(EPD)发布版。使用 matplotlib 所需的预打包库和所有必须的依赖软件（SciPy、NumPy、IPython 以及更多的其他软件包），均已包含在 EPD 发布版中。

matplotlib 以及与本书内容相关的软件，都可以使用常规的 Windows Installer 安装文件（*.exe）方式进行安装。

Python(x,y)（http://code.google.com/p/pythonxy/）是针对 Windows 32 位系统的免费科学计算项目，其中包含了 matplotlib 需要使用的依赖文件，它是在 Windows 系统上安装 matplotlib 的一种非常简单（而且是免费的）的方式。因为 Python(x,y)和 Python 模块安装器相互兼容，可以很容易地在 Python(x,y)基础上扩展安装其他 Python 库。在安装 Python(x,y)之前，系统应该没有安装 Python。

下面简短地说明一下如何使用预编译的 Python、NumPy、SciPy 和 matplotlib 二进制文件进行 matplotlib 的安装。首先，下载官方的 MSI 安装文件安装对应平台（x86 或 x86-64）的标准 Python 程序。之后，下载 NumPy 和 SciPy 的官方二进制文件并安装它们。在正确安装 NumPy 和 SciPy 之后，就可以下载最新稳定版 matplotlib 二进制安装文件并按照官方说明进行安装了。

1.5.3 补充说明

请注意，在 Windows 安装文件中 matplotlib 的示例相当有限。如果想尝试使用示例程序，可以下载并参考 matplotlib 源文件包中的 examples 子目录。

1.6　安装图像处理工具：Python 图像库（PIL）

Python 图像库（PIL）为 Python 提供了图像处理能力。PIL 支持的文件格式相当广泛，在图像处理领域提供了相当强大的功能。

快速数据访问、点运算（point operations）、滤波（filtering）、图像缩放、旋转、任意仿射转换（arbitrary affine transforms）是 PIL 中一些应用非常广泛的特性。例如，图像的统计数据即可通过 histogram 方法获得。

PIL 同样可以应用在其他方面，如批量处理、图像压缩、生成缩略图、图像格式转换以及图像打印。

PIL 可以读取多种图像格式，而图像写入支持的格式范围限定在图像交换和展示方面最通用的格式（有意为之）。

1.6.1　操作步骤

最容易也是最值得推荐的方式，是通过操作系统平台的包管理工具进行安装。

在 Debian/Ubuntu 系统中安装的命令如下。

```
$ sudo apt-get build-dep python-imaging
$ sudo pip install http://effbot.org/downloads/Imaging-1.1.7.tar.gz
```

1.6.2　安装过程说明

我们通过 `apt-get` 系统工具安装 PIL 所需的所有依赖软件，并通过 pip 安装 PIL 的最新稳定版本。一些老版本的 Ubuntu 系统通常不会提供 PIL 的最新发布版本。

在 RedHat/SciLinux 系统中，安装命令如下。

```
# yum install python-imaging
# yum install freetype-devel
# pip install PIL
```

1.6.3　补充说明

有一个专门针对 PIL 编写的在线手册。读者可以访问 http://www.pythonware.com/library/pil/handbook/index.htm 进行阅读，或是下载 PDF 版本：http://www.pythonware.com/media/data/pil-handbook.pdf。

Pillow 是一个 PIL 分支，它的主要目的是解决安装过程中的一些问题。Pillow 很容易安装，其网址为 http://pypi.python.org/pypi/Pillow。

在 Windows 平台上，也可使用二进制安装文件安装 PIL。从 http://www.pythonware.com/products/pil/下载.exe 安装文件，执行该文件将安装 PIL 到 Python 的 site-packages 目录。

如果需要在虚拟环境下使用 PIL，可手动将 PIL.pth 文件和位于 C:\Python27\Lib\site-packages 下的 PIL 目录复制到 virtualenv 的 site-packages 目录下。

1.7 安装 requests 模块

我们需要的大部分数据都可以通过 HTTP 或类似协议获得，因此我们需要一些工具来实现数据访问。Python 的 requests 库能让这部分工作变得轻松起来。

虽然 Python 提供的 **urllib2** 模块提供了访问远程资源的能力以及对 HTTP 协议的支持，但使用该模块完成基础任务的工作量还是很大的。

Request 模块提供新的 API，减轻了使用 Web 服务的痛苦，使其变得更直接。Requests 封装了很多 HTTP 1.1 的内容，仅在需要实现非默认行为的情况下才需要暴露相关内容。

1.7.1 操作步骤

安装 requests 模块最好的方式是使用 pip。安装命令如下。

```
$ pip install requests
```

也可以在 virtualenv 虚拟环境中执行安装命令，如果并不是所有项目都需要 requests，或是不同的项目需要使用不同版本的 requests。

为了更快地理解 requests 的功能，下面是一个使用 requests 的小例子。

```
import requests
r = requests.get('http://github.com/timeline.json')
print r.content
```

1.7.2 requests 使用说明

在本例中，我们向 www.github.com 站点的 URI 发送 HTTP GET 请求，以 JSON 格式返回了 GitHub 网站的活动时间表（也可以通过访问 https://github.com/timeline 得到 HTML 版本的活动时间表）。在成功读取 HTTP 响应后，对象 r 包含了 HTTP 响应内容以

及其他属性信息（HTTP 状态码、cookies、HTTP 头元数据，甚至包括当前响应所对应的请求信息）。

1.8　在代码中配置 matplotlib 参数

matplotlib 库提供了强大的绘图功能，是本书用的最多的 Python 库。在其配置文件即 .rc 文件中，已经为大部分属性设定了默认值。本节会介绍如何通过应用程序代码修改 matplotlib 的相关属性值。

1.8.1　准备工作

如前所述，matplotlib 配置信息是从配置文件读取的。在配置文件中可以为 matplotlib 的几乎所有的属性指定永久有效的默认值。

1.8.2　操作步骤

在代码执行过程中，有两种方式更改运行参数：使用参数字典（rcParams）或调用 matplotlib.rc() 命令。第一种方式中，可以通过 rcParams 字典访问并修改所有已经加载的配置项；第二种方式中，可以通过向 matlotlib.rc() 传入属性的关键字元组来修改配置项。

如果需要重置动态修改后的配置参数，可以调用 matplotlib.rcdefaults() 将配置重置为标准设置。

下面两段代码演示了之前介绍的功能。

使用 matplotlib.rcParams 的例子。

```
import matplotlib as mp
mpl.rcParams['lines.linewidth'] = 2
mpl.rcParams['lines.color'] = 'r'
```

使用 matplotlib.rc() 函数调用的例子。

```
import matplotlib as mpl
mpl.rc('lines', linewidth=2, color='r')
```

上面两个例子具有相同的语义。第二个例子中，我们设定后续的所有图形使用的线条宽度为 2 个点。第一个例子中的最后一条语句表明，语句之后的所有线条的颜色均为红色，除非用本地设置覆盖它，请看下面的例子。

```
import matplotlib.pyplot as plt
import numpy as np

t = np.arange(0.0, 1.0, 0.01)

s = np.sin(2 * np.pi * t)
# make line red
plt.rcParams['lines.color'] = 'r'
plt.plot(t,s)

c = np.cos(2 * np.pi * t)
# make line thick
plt.rcParams['lines.linewidth'] = '3'
plt.plot(t,c)

plt.show()
```

1.8.3 代码解析

首先，为了绘制正弦、余弦曲线，需要导入 matplotlib.pyplot 和 NumPy 模块。在绘制第一个图像之前，通过 plt.rcParams['lines.color']= 'r'语句显式地设置线条颜色为红色；接下来，对于第二个图像(余弦曲线)，通过语句 plt.rcParams['lines.linewidth'] = '3'显式地设定线宽为 3 个点。

如果需要重置设置，需要调用 matplotlib.rcdefaults()方法。

1.9 为项目设置 matplotlib 参数

本节介绍 matplotlib 使用的各种配置文件的位置，以及使用这些配置文件的意义。同时还将介绍配置文件中的具体配置项。

1.9.1 准备工作

如果不想在每次使用 matplotlib 时都在代码开始部分进行配置（像前一节我们做的那样），就需要为不同的项目设定不同的默认配置项。本节将介绍如何做到这一点。这种配置方式使得配置项与代码分离，从而使代码更加整洁。此外，你可以很容易在同事间甚至项目间分享配置模板。

1.9.2 配置方法

假设一个项目对于 matplotlib 的特性参数总会设置相同的值，就没有必要在每次编写新

的绘图代码时都进行相同的配置。取而代之的，应该是在代码之外，使用一个永久的文件设定 matplotlib 参数默认值。

通过 matplotlibrc 来配置文件，matplotlib 提供了对这种配置方式的支持。在 matplotlibrc 文件中包含了绝大部分可以变更的属性。

1.9.3 配置过程说明

配置文件可能存在于三个不同的位置，而它们的位置决定了它们的应用范围。这三个位置分别说明如下。

◆ **当前工作目录**：即代码运行的目录。在当前目录下，可以为目录所包含的当前项目代码定制 matplotlib 配置项。配置文件的文件名是 matplotlibrc。

◆ **用户级.matplotlib/matplotlibrc 文件(Per user .matplotlib/matplotlibrc)**：通常是在用户的$HOME 目录下（在 Windows 系统中，也就是 Documents and Settings 目录）。可以用 matplotlib.get_configdir() 命令来找到当前用户的配置文件目录。请参考随后的命令示例。

◆ **安装级配置文件(Per installation configuration file)**：通常在 python 的 site-packages 目录下。这是系统级配置，不过在每次重新安装 matplotlib 后，配置文件会被覆盖。因此如果希望保持持久有效的配置，最好选择在用户级配置文件中进行设置。对于笔者来说，目前对本配置文件的最佳应用方式，是将其作为默认配置模板。如果在用户级配置文件已经比较混乱，或者需要为新项目做全新配置时，可以基于该配置文件进行设置。

在 shell 中运行下面的命令，即可打印出配置文件目录的位置：

```
$ python -c 'import matplotlib as mpl; print mpl.get_configdir()'
```

配置文件包括以下配置项。

◆ axes：设置坐标轴边界和表面的颜色、坐标刻度值大小和网格的显示。

◆ backend：设置目标输出 TkAgg 和 GTKAgg。

◆ figure：控制 dpi、边界颜色、图形大小和子区（subplot）设置。

◆ font：字体集（font family）、字体大小和样式设置。

◆ grid：设置网格颜色和线型。

◆ legend：设置图例和其中文本的显示。

◆ line：设置线条（颜色、线型、宽度等）和标记。

◆ patch：是填充 2D 空间的图形对象，如多边形和圆。控制线宽、颜色和抗锯齿设置等。

◆ savefig：可以对保存的图形进行单独设置。例如，设置渲染的文件的背景为白色。

◆ text：设置字体颜色、文本解析（纯文本或 latex 标记）等。

◆ verbose：设置 matplotlib 在执行期间信息输出，如 silent、helpful、debug 和 debug-annoying。

◆ xticks 和 yticks：为 x、y 轴的主刻度和次刻度设置颜色、大小、方向，以及标签大小。

1.9.4 补充说明

如果你想了解前面提到的（和我们没有提到的）每个设置的详细信息，最好的方式是访问 matplotlib 项目的网站，那里提供了最新的 API 文档。如果需要获得进一步帮助，可以在用户和开发邮件组留言。本书最后还提供了一些有用的在线资源。

第 2 章
了解数据

在本章中，我们会介绍以下内容。

- 从 CSV 文件导入数据
- 从 Microsoft Excel 文件导入数据
- 从定宽数据文件导入数据
- 从制表符分隔的文件导入数据
- 从 JSON 数据源导入数据
- 导出数据到 JSON、CSV 和 Excel
- 从数据库导入数据
- 清理异常值
- 读取大块数据文件
- 读取流数据源
- 导入图像数据到 NumPy 数组
- 生成可控的随机数据集合
- 真实数据的噪声平滑处理

2.1 简介

本章涵盖了导入和导出各种格式数据的基本知识。除此之外，还包括清理数据的方式，比如值的归一化处理、缺失数据的添加、实时数据检查以及一些类似的技巧，以便正确地准备数据来进行可视化。

2.2 从 CSV 文件导入数据

在本节中，我们将处理每个人都能接触到的最常用的文件格式——CSV。顾名思义，CSV 是指逗号分隔的值（文件中还包括一个文件头，也是以逗号分隔的）。

Python 中有个 `csv` 模块支持读写各种方言格式的 CSV 文件。方言是很重要的，因为没有一个统一的 CSV 标准，不同的应用实现 CSV 的方式略有不同。当看到文件内容的时候，你往往就能很容易地辨认出文件使用的是哪种方言。

2.2.1 准备工作

在本节中，我们把 `ch02-data.csv` 文件的内容用作示例数据，你可以把它下载到本地。

我们假定示例数据文件和读取数据文件的代码在相同目录下。

2.2.2 操作步骤

下面的示例代码解释了如何从 CSV 文件导入数据，步骤如下。

1. 打开 `ch02-data.csv` 文件。
2. 首先读取文件头。
3. 然后读取剩余行。
4. 当发生错误时抛出异常。

读取完所有内容后，打印文件头和其余所有行。

```
import csv

filename = 'ch02-data.csv'

data = []
```

```
try:
    with open(filename) as f:
        reader = csv.reader(f)
    header = reader.next()
    data = [row for row in reader]
except csv.Error as e:
    print "Error reading CSV file at line %s: %s" % (reader.line_
num, e)
    sys.exit(-1)

if header:
    print header
    print '=================='
for datarow in data:
    print datarow
```

2.2.3　工作原理

首先，导入 csv 模块以便能访问所需的方法。然后，用 with 语句打开数据文件并把它绑定到对象 f。不必操心在操作完资源后去关闭数据文件，with 语句的上下文管理器会帮助处理。这在操作资源型文件时非常方便，因为它能确保在代码块执行完毕后资源会被释放掉（比如关闭文件）。

然后，用 csv.reader() 方法返回 reader 对象，通过该对象遍历所读取文件的所有行。在这里，每行内容不过是一个值列表，在循环中被打印出来。

文件的第一行是文件头，用来描述文件中每列的数据，在读取时多少有些不同。文件头并不是必需的，有些 CSV 文件就不带文件头，但是它们确实是提供数据集合的最小元数据信息的一个不错的方式。然而，有时候会碰到用分隔的文本或者仅用作元数据的 CSV 文件，来描述数据格式和附加数据的情况。

此时只能打开文件来看看第一行是数据头还是数据（例如查看文件的前几行）。这在 Linux 系统上用 bash 命令如 head 可以很容易做到，格式如下所示。

```
$ head some_file.csv
```

在遍历数据时，我们把第一行存储为文件头，把其他行添加到数据列表中。

读取文件时一旦出了问题，csv.reader() 方法会生成错误信息。为了能帮助用户发现问题，可以捕获这些错误信息并给用户打印出有用的信息。

2.2.4 补充说明

如果想了解 csv 模块的来龙去脉，可以看一下 PEP 文档中的《CSV 文件 API》，参见 http://www.python.org/dev/peps/pep-0305/。

如果想加载大数据文件，明智的做法通常是使用一些著名的库如 NumPy 的 loadtxt() 方法，这个方法可以很好地处理 CSV 大数据文件。

基本用法非常简单，如下面的代码段所示。

```
import numpy
data=numpy.loadtxt('ch02-data.csv',dtype='string', delimiter=',')
```

值得注意的是，为了能让 NumPy 正确地分隔数据，需要定义分隔符。numpy.loadtxt() 方法比类似的 numpy.genfromtxt() 方法要快一些，但是后者能更好地处理缺失数据，而且在处理已加载文件的某些列时，可以使用一些方法来做些额外的事情。



目前，在 Python 2.7.x 版本中，csv 模块不支持 Unicode 编码，必须把读取的数据显式地转换成可打印的 UTF-8 或者 ASCII 编码。官方的 Python CSV 文档提供了一些解决数据编码问题的很好的示例。

Python3.3 及后续版本默认支持 Unicode 编码，不存在此类问题。

2.3 从 Microsoft Excel 文件中导入数据

虽然 Microsoft Excel 支持一些画图操作，但如果需要更加灵活和强大的可视化效果，就需要把数据从表单中导出到 Python 中以备将来之需。

从 Excel 文件导入数据的通常做法是把数据从 Excel 中导出到 CSV 格式的文件中，然后用上节中提到的方法使用 Python 从 CSV 文件中导入数据。如果只有一两个文件（并且安装了 Microsoft Excel 或者 OpenOffice.org），事情就相当简单。但是如果想自动化地对大量文件进行数据管道处理（作为数据连续处理流程的一部分），那么手动把每个 Excel 文件转换成 CSV 文件的做法就行不通了。因此，我们需要一种方法来读取 Excel 文件。

通过 www.python-excel.org 项目提供的软件包，Python 可以很好地支持 Excel 文件的读写操作。对读操作和写操作的支持是通过不同模块实现的，而且是平台无关的。换言之，我们不必为了读取 Excel 文件而必须要在 Windows 平台上工作。

Microsoft Excel 文件格式随着时间发生着变化，不同的 Python 库对其都有相应的支持。在写作本书时，XLRD 最新的稳定版本是 0.90，它已经支持读取 .xlsx 文件了。

2.3.1　准备工作

首先，我们需要安装所需的模块，在这个例子中我们将使用 xlrd 模块。我们将用 pip 在虚拟环境中安装此模块。

```
$ mkvirtualenv xlrdexample
(xlrdexample)$ pip install xlrd
```

安装完毕后，我们将用 ch02-xlsxdata.xlsx 示例文件做演示。

2.3.2　操作步骤

接下来的示例代码将展示如何从已知的 Excel 文件中读取一个样本数据集合。

1. 打开文件的工作簿。
2. 根据名称找到工作表。根据行数（nrows）和列数（ncols）读取单元格的内容。
3. 因为只是用作演示，本例仅打印出了读取的数据集合。

```
import xlrd

file = 'ch02-xlsxdata.xlsx'

wb = xlrd.open_workbook(filename=file)

ws = wb.sheet_by_name('Sheet1')

dataset = []

for r in xrange(ws.nrows):
    col = []
    for c in range(ws.ncols):
        col.append(ws.cell(r, c).value)
    dataset.append(col)

from pprint import pprint
pprint(dataset)
```

2.3.3　工作原理

让我们试着解释一下 xlrd 模块使用的简单对象模型。在最上层是一个包含一个或多个工作表（xlrd.sheet.Sheet）的工作簿（Python 类 xlrd.book.Book）。每个工作表有一个单元格对象（xlrd.sheet.Cell），我们能从单元格中将值读取出来。

通过调用 open_workbook() 方法，我们从文件中加载了一个工作簿，并返回一个 xlrd.book 实例。Book 实例包含了一个工作簿的所有信息，如工作表单。通过调用 sheet_by_name() 方法可以访问指定的工作表，如果需要所有的工作表，可以调用 sheets() 方法。sheets() 方法返回一个 xlrd.sheet.Sheet 实例的列表。xlrd.sheet.Sheet 类有行和列属性，我们能通过这些属性来指定循环的范围，并通过调用 cell() 方法来访问工作表中的每个特定的单元格。虽然有一个 xrld.sheet.Cell 类，但并不需要直接使用它。

请注意，日期是以浮点数而不是以某个日期类型存储的。但是，xlrd 模块有能力检查数据的值，并推断出数据值实际上是否为一个日期。这样，我们就能通过检查单元格类型来得到 Python date 对象。如果数字的字符串像日期，xlrd 模块将返回 xlrd.XL_CELL_DATE 作为单元格类型。这里用一段代码来说明这点：

```
from datetime import datetime
from xlrd import open_workbook, xldate_as_tuple
...
cell = sheet.cell(1, 0)
print cell
print cell.value
print cell.ctype
if cell.ctype == xlrd.XL_CELL_DATE:
    date_value = xldate_as_tuple(cell.value, book.datemode)
    print datetime(*date_value)
```

这个日期字段还有些问题。因此，如果需要针对日期做大量的工作，请参见官方文档和邮件列表。

2.3.4 补充说明

xlrd 模块的一个非常好的特性是它能按照需要仅加载文件的部分内容到内存中。open_workbook 方法有一个 on_demand 参数，在调用时把它置为 True，工作表就能按需加载了。例如：

```
book = open_workbook('large.xls', on_demand=True)
```

本节没有提到 Excel 文件的写操作。一部分原因是后面会有单独的一节去讲述它，另一部分原因是 Excel 的写操作需要另一个不同的模块——xlwt 来完成。你能从本章的“导出数据到 JSON、CSV 和 Excel”一节获得更多的信息。

如果需要一些在前面介绍的例子和模块中没有涉及的特定用法，PyPi 上有一个操作工作

表的其他一些Python模块的列表，也许能对你有帮助，请访问http://pypi.python.org/pypi?:action=browse&c=377。

2.4　从定宽数据文件导入数据

事件的日志文件和基于时间序列的文件是数据可视化中最常见的数据源。有时候，可以以制表符分隔数据这种CSV方言来读取它们，但有时它们不是通过任何特殊字符分隔的。实际上，这些文件中的字段是有固定宽度的，我们能通过格式来匹配并提取数据。

一种做法是逐行读取文件，然后用字符串操作方法把字符串分割成独立的部分。这种做法比较直接，如果性能不是问题的话可以作为首选。

如果性能更重要，或者要解析的文件非常大（几百兆字节），用Python中的`struct`模块（http://docs.python.org/library/struct.html）能提升性能，因为这个模块是用C语言而不是Python实现的。

2.4.1　准备工作

因为`struct`模块是Python标准库的一部分，所以不必安装额外的软件来完成本节的内容。

2.4.2　操作步骤

我们将会使用一个预先生成的数据集合，其中有一百万行定宽数据记录。样本数据格式如下：

```
...
207152670 3984356804116 9532
427053180 1466959270421 5338
316700885 9726131532544 4920
138359697 3286515244210 7400
476953136 0921567802830 4214
213420370 6459362591178 0546
...
```

这个数据集合是通过代码生成的，代码文件`ch02-generate_f_data.py`可以在本章的代码库中找到。

现在可以读取数据了。示例代码如下，步骤如下。

1. 指定要读取的数据文件。
2. 定义数据读取的方式。

3. 逐行读取文件并根据格式把每行解析成单独的数据字段。

4. 按单独数据字段的形式打印每一行。

```
import struct
import string

datafile = 'ch02-fixed-width-1M.data'

# this is where we define how to
# understand line of data from the file
mask='9s14s5s'

with open(datafile, 'r') as f:
    for line in f:
        fields = struct.Struct(mask).unpack_from(line)
        print 'fields: ', [field.strip() for field in fields]
```

2.4.3 工作原理

可以用 head、more 或者类似的 Linux shell 命令来查看文件内容，然后根据所见的数据文件格式定义掩码格式。

字符串格式用来定义要提取的数据的期望显示格式。我们用格式字符定义数据类型。因此，如果掩码定义为 9s15s5s，我们可以读作“9 个字符宽度的字符串，跟着一个 15 个字符宽度的字符串，再跟上一个 5 个字符宽度的字符串。”

一般来说，c 定义为字符（C 语言中的 char 类型）或者长度为 1 的字符串，s 定义为字符串（C 语言中的 char[]类型），d 定义为浮点数（C 语言中的 double 类型），以此类推。在 Python 官方网站上有完整的对应表，参见 http://docs.python.org/library/struct.html#format-characters。

然后逐行读取文件内容并根据指定的格式解析（通过 unpack_from 方法）每一行。因为在字段前面（或者后面）可能有多余的空格，用 strip()方法可以去掉每个字段的前导和后导空格。

对于解包，可以使用 struct.Struct 类的**面向对象（object-oriented, OO）**的方式，但也可以像下面的代码这样使用非面向对象的方式：

```
fields = struct.unpack_from(mask, line)
```

两种方式唯一的不同是使用的模式。如果想用相同的格式化掩码执行更多的操作，面向对象的方法可以不必在每次调用时声明格式。而且，它让我们有能力在将来继承 struct.Struct 类，为特定需求进行扩展或者提供额外的功能。

2.5　从制表符分隔的文件中读取数据

另一种常见的平坦数据文件（flat datafile）格式是制表符分隔的文件。它可能导出自 Excel 文件，也可能是一些定制软件的输出。

庆幸的是，通常我们可以按与 CSV 文件几乎相同的方式来读取这种格式的文件内容。因为 Python 的 `csv` 模块支持的方言能让我们用相同的原则来读取相似文件格式的变体——其中一种就是制表符分割格式。

2.5.1　准备工作

此时假定我们已经知道如何读取 CSV 文件。如果还不清楚，请先参见 2.2“从 CSV 文件导入数据”一节。

2.5.2　操作步骤

我们将重用 2.2“从 CSV 文件导入数据”一节中的代码，在这里只需要改动一下使用的方言。

```
import csv

filename = 'ch02-data.tab'

data = []
try:
    with open(filename) as f:
        reader = csv.reader(f, dialect=csv.excel_tab)
    header = reader.next()
        data = [row for row in reader]
except csv.Error as e:
    print "Error reading CSV file at line %s: %s" % (reader.line_num, e)
    sys.exit(-1)

if header:
    print header
    print '====================='

for datarow in data:
    print datarow
```

2.5.3　工作原理

除了实例化 `csv` 读对象的一行代码不同，上述代码和在“从 CSV 文件导入数据”一

节中的非常相似。在那行代码中，我们指定 dialect 参数为 excel_tab 方言。

2.5.4 补充说明

基于 CSV 格式读取数据的方式没有办法处理有“脏数据”的情况。换言之，如果有几行不是仅以换行符结尾，而是有多余的\t（制表符）标记，这时就需要在切分前对特殊行的数据进行单独清理。ch02-data-dirty.tab 是含有“脏数据”的制表符分隔的文件，下面的示例代码在读取文件数据时对“脏数据”进行了清理：

```
datafile = 'ch02-data-dirty.tab'

with open(datafile, 'r') as f:
    for line in f:
        # remove next comment to see line before cleanup
        # print 'DIRTY: ', line.split('\t')

        # we remove any space in line start or end
        line = line.strip()

        # now we split the line by tab delimiter
        print line.split('\t')
```

我们看到了另一种分隔字段的方式——使用 split('\t')方法 。

与使用 csv 模块的方式相比，split()方法的优势有时候体现在：仅仅通过改变方言就可以重用相同的代码来读取数据。至于如何检测方言，可以根据文件扩展名（.csv 和.tab）或者其他一些方法（比如使用 csv.Sniffer 类）来判断。

2.6 从 JSON 数据源导入数据

本节将展示如何读取 JSON 格式的数据。此外，我们将会使用一个远程数据源。这会让本节的内容有点复杂，但同时也会使其更加实用，因为在现实世界中，我们会更多地遇到远程数据源，而不是本地数据。

JavaScript Object Notation（JSON）作为一种平台无关的格式被广泛地应用于系统间或者应用间的数据交换。

本文中，资源是我们可以读取的任何东西，可以是一个文件或者一个 URL 端点（可以是远程进程/程序的输出，或者一个远程静态文件）。简言之，我们不关心谁产生了数据源以及是怎么产生的，我们只需要它是一种已知的格式，如 JSON。

有关此电子图书的说明

本人由于一些便利条件，可以帮您提供各种中文电子图书资料，且质量均为清晰的PDF图片格式，质量要高于网上大量传播的一些超星 PDG 的图书。方便阅读和携带。只要图书不是太新，文学、法律、计算机、人文、经济、医学、工业、学术等方面的图书，我都可以帮您找到电子版本。所以，当你想要看什么图书时，可以联系我。我的 QQ是：**89039855**，大家可以在 QQ上联系我。

此 PDF 文件为本人亲自制作，请各位爱书之人尊重个人劳动，敬请您不要修改此 PDF文件。因为这些图书都是有版权的，请各位怜惜电子图书资源，不要随意传播，否则，这些资源更难以得到。